

矮人要塞引擎架构与性能优化研究

设计理念与引擎结构

《矮人要塞》（Dwarf Fortress）以其极高的模拟深度著称，其核心设计理念是在庞大细致的世界中模拟方方面面的细节，而不以现代3D图形为重点。游戏采用矩形网格的世界表示，每个游戏刻（tick）都会更新世界中所有实体的状态，包括矮人、动物、怪物、液体流动、天气变化等。这种**离散时间步进**的更新机制意味着每一帧游戏都会遍历大量对象并执行复杂运算。

引擎技术上，由Tarn Adams使用C/C++自研（代码超过70万行，20年积累）。图形和音频方面利用了OpenGL、SDL库和FMOD等。长期以来，模拟计算主要在**单线程**上完成，只有绘图/界面处理在独立线程中进行。正如Adams所言：“如果说异步就是多线程，那么除了图形显示本身外，我们没有做任何这样的事”。因此，矮人要塞的大部分计算都发生在一个核心上，这也使得游戏性能高度依赖CPU的单核速度和缓存效率。游戏默认将模拟帧率封顶为每秒100帧（FPS），以保持游戏节奏可控，实际帧率会随着要模拟的内容多少而动态下降。

矮人要塞的**模拟深度**极为惊人：每个矮人都有详细的生理和心理模型（器官、情绪、喜好等），世界会生成悠久历史、文明和传说，并在运行中模拟水流、温度传导、毒素、天气、战斗碰撞和损伤等。这种对细节的执着意味着**庞大的计算负荷**。例如，游戏在2008年增加Z轴变为三维空间时，开发者花费数周修改了所有依赖XY坐标的逻辑以适应新的Z维度。如此复杂的模拟在当年硬件上只能通过精心优化的单线程逻辑实现。总体而言，矮人要塞引擎架构体现了**牺牲硬件利用率来换取模拟一致性**的思路：由一个核心串行地处理几乎所有游戏逻辑，以避免并发带来的不确定性和bug，但也因此容易受限CPU性能。

常见性能瓶颈与卡顿原因

随着堡垒发展，玩家常会遭遇“FPS死亡”（帧率骤降）。主要原因包括：

- **活动单位数量：单位AI更新**是性能的最大开销来源。游戏中每个生物（矮人、宠物、敌人等）在每个刻都要决策行动、感知环境、更新状态。当单位总数增加时，CPU时间被大量消耗在这些更新上。据分析，在较大的要塞中，**单位轮流行动占用了超过60%的处理时间**。这还包含了单位间的**视线(Line-of-Sight, LOS)判定**等逻辑。当同屏或同区域聚集许多生物时，LOS计算呈 $O(n^2)$ 增长，需要对检查是否互相可见，是**当前版本中最耗时的部分**。即使v50.05版本对视线算法进行了优化，这仍是主要瓶颈之一。因此，大量单位（如上百矮人或敌军来袭）往往导致严重的帧率下滑。开发者也提醒**限制人口数量**（通过 `d_init.txt` 设置人口上限）是防止FPS下降的有效手段。同样，限制**入侵敌人**的波次规模或直接关闭侵袭也能减少负荷。
- **路径搜索：路径查找**（pathfinding）是矮人要塞中较复杂的算法任务。每当生物寻找路线去执行任务时，游戏在三维网格中进行A搜索。这在迷宫般的三维空间（尤其是开放的地下洞穴）里开销很高。算法使用了一种**Chebyshev距离启发**，但在有上下层连通时可能需要广泛的搜索。若有单位反复找不到路径（如被困地点尝试路径又失败），CPU会浪费大量循环在无效搜索上，导致突然的卡顿。社区将这类情况称为典型的路径问题卡顿：例如有生物掉入陷阱或困在地形中，大量路径请求无法完成就会拖慢游戏。此外，路径空间越大（地图/据点越大、挖空区域越多），每次搜索需要考虑的节点就越多，也降低性能。玩家经验显示，一些设计细节会影响路径效率，比如过于密集的楼梯设计会增加路径分支（3×3通天楼梯每层提供多达9个出口方向），而使用3格宽的斜坡/坡道可能限制选择使路径计算更简单。不过需要指出，*Profiling*数据显示路径算法本身通常占用不到总帧时间的10%——也就是说，相比起单位AI和

其他系统，路径搜索并非绝对主导瓶颈。但在极端情况下（如开启大面积洞穴、单位遍历整图），路径计算会放大帧率问题，因此通过封闭多余区域、规划交通优先级*等方式减轻路径搜索工作量，依然能略微提升性能。

- **物品和垃圾累积**：随着要塞发展，会产生海量物品（矿石、木材、工艺品、尸体、衣物等）。物品数量本身也会影响性能。一方面，每件物品在运行中可能参与**温度传递**、**磨损计时**等计算；物品越多，这些遍历检查越慢。特别是开启温度时，游戏需要对地图上每件可受温度影响的物品每帧检查热量变化，这会造成显著开销。此外，每件装备的**磨损**（wear）状态也会定期更新，大量穿在矮人身上的旧衣服增加了CPU负担。另一方面，**海量物件影响内存占用和缓存**，比如有品质或装饰的物品需要存储更多数据，在数量超过一万件后，会明显拖累循环处理和内存访问。虽然**单个栈(stack)内物品数量**对帧率影响不大，但**物品堆的数量**会通过增加搬运任务、库存搜索等间接影响性能^①。因此，任由废弃物遍地、仓库爆满会加剧卡顿。有玩家统计发现，当物品总数达数万时，游戏速度显著下降。**污染物**（血迹、泥浆、呕吐物等地面污渍）也属于特殊的物品，会不断触发计算且存在bug，过多污染会降低FPS。总之，**物品清理不及时**是后期常见的隐性能杀手。
- **液体流动与物理更新**：矮人要塞模拟现实中的液体动力学，包含水和熔岩的流动、压力和蒸发冻结。这些**液体流体计算**相当耗费CPU。当有大量流动水（例如瀑布、灌溉渠、洪水）时，游戏每帧都要更新相邻格的液面高度并重新计算流动路径，导致卡顿或尖峰式的**瞬时掉帧**^②。特别是**大面积的熔化/冻结事件**（如春季大规模融雪）会使帧率骤降^②。开发者提示水位变化会引发**连接组件重绘**和**通路重计算**等昂贵操作，应尽量避免无节制的流水机制。玩家经验也表明，**持续运行的喷泉或薄雾发生器**（mist generator）等装置会长期占用性能。因此液体相关机制是游戏著名的性能瓶颈之一，需要谨慎使用或封闭管理。
- **AI逻辑与其他系统**：除了上述主要因素，游戏中的许多**后台逻辑**都会占用CPU时间。例如：矮人的**思考和情绪更新**（每个矮人有复杂的心情和记忆系统，需要定期处理）、**工作任务调度**（游戏为每个矮人分配工作，要扫描可用资源和路径）、**战斗计算**（攻击判定、伤口处理）等。社区玩家提到，某些劳动指令如果不加选择地赋予所有矮人，比如“采集植物”（全员扫描地图上的可收获植物），会让CPU负荷激增，最好限定少数矮人从事。另外，**温度与天气**也是全局系统：每帧都会计算温度扩散、着火判定、天气效果（雨水积聚冲刷污迹等）。这些系统在开启时提供了拟真的环境，但也各自增加了一份周期性计算量。关闭温度后，比如火焰不再传播、熔岩冷却停止等，虽然牺牲真实感但能显著提升效率。总的来说，矮人要塞的卡顿往往是**累积效应**：单位、路径、物品、流体、环境等多个系统叠加，使得随着要塞规模扩大，帧率不可避免地逐步下降。

作者官方观点与未来改进方向

面对性能问题，开发者Tarn Adams也多次给出解释并进行改进。他强调**保持模拟深度的同时尽可能优化**。在过去很长一段时间里，矮人要塞没有采用多核并行，这既有历史代码架构原因，也因开发者对多线程并不熟悉，担心并发会引入大量Bug。Tarn直言自己“**没有这方面经验，而且多线程容易出Bug**”。因此早期他倾向于通过**代码层面的优化**来提升性能，而非架构性的重构。例如，他遵循一个原则：“想想某件事实际需要多频繁执行，尽量只按需执行”。这体现在许多细节上：**跳帧与延迟更新**——并非每个逻辑每帧都算，而是能隔帧或定期计算的就延长间隔；**惰性计算**——能缓存的结果就缓存，需要时才刷新。

具体例子包括：在路径算法上，他不采用标准教材那样维护完整的“已访问节点列表”，而是利用地图已有结构标记节点以避免额外的数据维护开销。又如，在物品系统中，他为物品设置了“可能包含其他物品”的标志位。如果某件物品永远不会容纳别的物品（比如石块不会包含其他对象），就可跳过大量检查，大大减少遍历调用。“牺牲一点点内存（每个物品一个比特标志），换取大量检查的省略，整体是值得的”。类似这种**通过标志和数据结构调整**来剪枝运算的技巧，广泛用于矮人要塞各系统中。

然而，Tarn也承认自己在**现代CPU优化**方面有所不足，例如**缓存利用和分支预测**等底层性能技巧上不够讲究。随着时间推移，他开始接受社区和合作伙伴的帮助来改进性能。值得关注的是，**Steam发行版本**对引擎进行了重大升级：**引擎从SDL1升级到SDL2**，并引入了**64位支持**和诸多优化。Steam版要求64位系统，这解决了旧版内存上限问题，并利用现代系统性能。更重要的是，**官方开始尝试多线程**：在2023年6月发布的v50.09版中，加入了**实验性的多线程选项**，可在游戏设置中开启。据Tarn发布说明，这次升级“带来了许多优化，并提供了一个（实验性的）多线程选项，使游戏运行更快”。这标志着矮人要塞正式踏出了多线程的第一步。这部分工作主要由社区资深黑客Putnam协助完成。玩家反馈多线程模式下帧率有一定提升，但由于实验性质，稳定性和收益因场景而异。

此外，Tarn在开发日志中也提及了一些**未来的性能改进方向**。例如，他计划逐步**优化视线计算、改进生物AI调度**，尽量减少不必要的全局扫描。视线系统已在近期版本优化过一次，但仍有进一步提升空间（例如引入空间分区来减少单位彼此判定次数）。对于并行化，Tarn表示“**社区甚至提供了微线程库让我尝试**”，他意识到很多模拟环节其实可以并行（比如独立生物的决策），只是需要重构代码。在Steam版后续更新中，我们可能会看到**更多子系统移入多线程**（例如将部分路径计算、流体更新放到并行线程）。不过Tarn也强调，矮人要塞的开发重心仍是**丰富游戏内容**（如神话与魔法系统的实现），性能优化会作为持续的背景任务贯穿开发过程，而不会大幅削减模拟细节来换取性能。总体而言，官方态度是在保持游戏特色的前提下“**拾遗补缺**”式地优化：该用的新技术会用，但不会为性能牺牲模拟真实度，宁可通过选项让玩家自行取舍（如温度、天气可关闭）。

社区总结的实用优化技巧

多年来，英文社区（Bay12论坛、Reddit、Steam讨论区）玩家集思广益，摸索出大量提升帧率的窍门。这些技巧大多围绕**减少游戏需处理的对象数量**和**降低算法复杂度**两大方向：

- **控制地图与区域规模**：缩小Embark地图尺寸是公认最有效的提升FPS方法之一。默认4×4区块的地图对CPU负担较大，而缩减为3×3甚至2×2区域，会极大减少需要模拟的地形大小。2×2地图面积仅为4×4的1/4，在三维上依然足够建造一个普通要塞，但性能改善显著。此外，**减少地下层数和洞穴**也有帮助。世界生成时可以降低洞穴层数（默认3层）或高度开放度，以减少地下生物和路径复杂性。很多玩家会在世界生成高级参数中关闭部分深层内容（如关闭冗余的洞穴层、熔岩海），换取更少的背景模拟开销。
- **限制人口与生物数量**：正如前文所述，大量单位是性能杀手，因此**主动控制单位数量**非常关键。具体做法包括：在初始化文件中设置合理的**矮人移民上限**，例如将人口封顶在50或80而非默认为100+。**宠物和牲畜**也需要控制繁殖，否则数百只动物漫无目的地游荡会拖慢游戏。许多玩家选择定期屠宰过剩的幼崽、阉割宠物防止无限繁殖，维持动物数量在可控范围（如同社区戏称的“猫汤”梗：过多的猫会导致严重卡顿，煮猫汤是无奈之举）。对于敌人入侵，可在 `d_init` 设置中调低**侵袭上限**或关闭侵袭，以免来敌过于庞大让游戏陷入低帧。此外，**封闭地下洞穴**能够阻止地下生物不断生成与路径寻找，这实际上减少了无关单位的活动范围。总之，“单位少=帧率高”是基本原则。
- **优化要塞布局以减轻路径开销**：社区玩家总结出很多建筑规划上的技巧来优化路径计算。常用方法有：**封锁无用区域**，使用墙壁或桥封住不再使用的矿坑、隧道，缩小可通行空间，这样路径算法不必考虑那些区域。**利用交通区划(Traffic Designations)**，在 `d` 菜单中设置高流量区和低流量区，引导路径算法优先搜索主要通道，减少在偏僻房间内浪费时间。例如，将主走廊设为“高”流量、各偏僻死角房间设为“低”流量，可以加速寻路。**分散楼梯或采用斜坡**：很多新人喜欢用3×3密集垂直楼梯，但资深玩家发现这种布局虽然方便上下，但寻路时可选节点过多，算法负担重。一些玩家建议改用**宽阔的坡道或分隔开的多组楼梯**，减少每层节点分支，有助于路径计算。还有人建议**避免挖掘超大空间**（如贯通几十层楼高的中庭），因为过大的开放空间也让寻路时需要考虑的区域变大。合理规划要塞，使各功能区紧凑排列、垂直相邻，可以缩短通勤路径，从而在一定程度上提升效率。

- **减少物品与垃圾**：为避免物品堆积拖垮帧率，玩家开发了多种**清理和压缩库存**的方法。“矮人原子碎石机”(Dwarven Atom Smasher)是一个经典技巧：建造一座可开合的桥，当作垃圾压实机，把不要的物品丢在桥下反复开关，利用桥的粉碎特性直接销毁物品。这可以大量消除碎石、尸体等垃圾。“量子库存”(Quantum Stockpile)则是利用矿车漏洞将无数物品堆叠在同一格，实现无限压缩存储，从而腾出地图空间、减少遍历开销。不过需要注意，量子堆放仅在不查看仓库时有利于性能，打开那个区域界面仍需绘制所有物品。**定期交易/丢弃**：玩家也建议定期把过剩物资打包卖给商队，或扔进岩浆坑销毁，以免要塞库存爆炸性增长。**自动清洁污染**：血迹等可通过引水冲刷要塞各层来清洁，避免污染物无限堆积。此外，游戏更新后闲置很久的旧衣服会产生垃圾警告，可通过指派回收或直接销毁处理旧衣，以免遍地破衣服降低FPS。简而言之，“勤于整理”是高帧率的秘诀：保持要塞整洁、物尽其用，才能让循环计算轻装上阵。
- **关闭或减少特定计算**：对于不介意削弱部分真实性的玩家，有几个设置可以极大提升帧率。**关闭温度(TEMPERATURE)**是效果最显著的优化之一，许多玩家报告FPS提高了一倍甚至更多。关闭温度后，火和熔岩等行为会异常（火焰不传播、熔岩不会冷却成黑曜石等），但在没有大量涉及温度的工程时这是值得的权衡。**关闭天气(WEATHER)**也能略微提升速度——雨水将不再下落和蒸发，但要注意雨不会洗掉污垢或填满积水池了。**调低帧率设置**：在 `init.txt` 中有 `G_FPS` 等参数控制绘图频率，适当降低界面刷新率可以减少CPU用于渲染/输入检测的占用，将更多资源留给模拟计算（渲染线程与模拟线程分离，所以降低绘图FPS不会加快模拟本身，但可以减轻系统整体负荷）。还有玩家会**暂时关闭AI密集任务**：比如大规模战役时暂停平民工作，避免平民AI抢占CPU；或者当FPS已经很低时，尽量不指定大量挖掘和建筑任务以免进一步雪崩。以上措施相当于**用配置换性能**，玩家可根据需要灵活运用。
- **硬件与系统优化**：由于矮人要塞主要吃单线程CPU，为其优化硬件的策略也与普通3D游戏不同。升级显卡几乎无益处；反而**提高CPU主频和增大高速缓存**能显著改善游戏性能。社区推测DF的瓶颈在于缓存未命中，因此带有超大L3/L4缓存的CPU、有快速低延迟内存的系统会运行更流畅³。有技术玩家在Linux下尝试将DF整个运行于 `tmpfs` 内存盘中，或替换内存分配器为更高效的 `mimalloc/jemalloc`，据称也获得了一定帧率提升。此外，在多任务环境下，提升DF进程的优先级（在类Unix系统用 `renice -n -20`）可确保CPU优先服务游戏线程，也可以缓解卡顿。

总的来说，社区的经验之谈可以归结为一句话：“**减少游戏需要处理的事物**”。无论是限制生物、物品，还是缩小空间、降低频率，目标都是让每游戏刻的计算量降下来。通过上述各种方法，老玩家们常能把一个中大型要塞的FPS从垂死的10提升回20-30，延长要塞的寿命。

DFHack插件和Mod的帧率改进方案

除了手动优化，玩家还利用DFHack等工具和Mod来提升矮人要塞的帧率与稳定性。DFHack是社区开发的内存修改工具集，提供许多方便的命令，其中一些专门用于性能维护：

- **自动清理与丢弃**：`autodump` 命令可批量倾倒或摧毁物品，将选定物品瞬间移动到垃圾区或直接删除。这在处理成千上万块碎石、一地尸体时非常好用。`cleanowned` 命令则会没收并丢弃矮人**拥有但废弃的物品**（如破烂衣服）。很多玩家定期运行该命令清理衣柜，从而减少磨损检查和矮人思念旧物的计算。当没有替换衣物时此操作可能触发不满，但相比卡顿这是小代价。
- **清除污染**：`clean` 和 `spotclean` 命令可以移除地图上的血迹、泥污等**污染层**。这些污染物正常情况下需要矮人清扫或雨水冲刷，但利用DFHack可以一键抹除，从而减少因污染导致的FPS下降（有已知Bug导致过多污染降低性能）。定期 `clean all` 可保持世界干净利落。

- **液体检查：** `flows` 命令用于统计当前地图上有液体流动的区块数。它帮助玩家识别哪里存在大量流水导致性能问题。玩家据此可以针对性地修建水闸、封闭水路或暂停抽水泵，以降低持续流动液体的数量。
- **温度优化：** `tweak fast-heat` 是DFHack提供的一个调整，用于加速温度更新的算法。启用此调整后，温度传播和冷却计算会更高效，从而缓解因开启温度导致的性能损失。
- **时间流速调整：** `timestream` 命令相当特别，它能更改游戏的时间流逝倍率，使低FPS情况下游戏进程加速。例如，`timestream -fps 100 -units` 可以让实际20FPS时的游戏按100FPS的速度推进，通过加快日历和单位行动，让玩家主观感觉速度变快。这并非真正提高底层帧率，而是改变游戏速度尺度来延缓“龟速”的挫败感，对后期FPS垂危的堡垒十分有用。
- **矮人加速与瞬移：** `fastdwarf` 命令可以令所有矮人和生物移动、干活速度加倍，甚至直接瞬移。虽然这是作弊性质的，但很多玩家单机自娱时无碍伦理。开启后矮人工作效率成倍提高，等同于缩短了等待时间，间接提升了玩家体验上的“帧率”。不过这对实际CPU负载没有帮助，只是减少了任务完成所需的刻数。

除了DFHack，Mod也可优化性能。一些Mod通过改动游戏原始数据，减少某些机制的负担。例如Modest Mod提供了“永恒时尚（Eternal Fashion）”模块，将衣物设定为不磨损，从而消除衣物老化检查对性能的影响。还有Masterwork等大型Mod允许制造金属衣服，替代会腐烂磨损的布衣，以减少垃圾衣物产生。部分Mod调整了生物产出频率、降低动物繁殖率等，间接减轻了后期负荷。此外，社区曾有人尝试过多线程改造的插件，但由于矮人要塞闭源且结构复杂，这类项目鲜有成功，大多停留在理论或概念验证阶段。如今官方已经开始提供实验性多线程支持，未来可能会有Mod配合优化调度。

总之，借助DFHack和Mod，玩家可以对游戏做“外科手术式”的性能改进：清理世界垃圾、跳过不必要的计算，以及加快游戏节奏。这些工具对于经营多年、规模庞大的要塞特别有价值，常常能够把原本濒临放弃的存档“抢救”回来，再续辉煌。

结语：矮人要塞的性能瓶颈源自其无与伦比的模拟深度和多年历史包袱。单线程架构、海量实体和复杂算法使其在大型要塞时难免出现卡顿。但正如Tarn Adams所坚持的，这是模拟真实性的代价。所幸，通过官方的持续优化和社区的聪明智慧，我们可以采取多种措施来延缓“FPS死亡”。从限制规模、精心布局，到借助DFHack清道夫和实验性多线程支持，矮人要塞依然可以在“欢乐血崩”的同时尽可能流畅地运行下去——毕竟，对矮人而言，快乐地卡顿，总好过无趣地流畅。

参考资料：

1. Dwarf Fortress Wiki – *Maximizing framerate*, detailed guide on performance optimization等
2. Bay 12 Forums – 社区讨论帧率因素和优化建议
3. Stack Overflow Blog – *How Dwarf Fortress is built*, Tarn Adams访谈（代码架构与多线程）
4. Gamedeveloper采访 – *How Tarn Adams upgraded and optimized Dwarf Fortress...*（Steam版优化心得）
5. Dwarf Fortress Wiki – *System requirements*, 关于硬件瓶颈与缓存影响
6. DFHack 文档 – 提供的提升FPS相关命令说明
7. Bay 12社区及Steam论坛 – 玩家实测与官方声明合集等

3 System requirements - Dwarf Fortress Wiki

https://dwarffortresswiki.org/index.php/System_requirements